Um **Annotation Processor** (ou Processador de Anotações) em Java é uma ferramenta integrada ao compilador (`javac`) que escuta, analisa e processa anotações presentes no código-fonte durante a **fase de compilação**. [1]

Como funciona

1. **Varredura:** Durante a compilação do projeto, o compilador verifica as classes, métodos e atributos em busca de anotações específicas.
2. **Execução:** O processador (uma classe que geralmente estende `AbstractProcessor`) lê esses metadados.
3. **Geração de Código:** Ele pode criar **novos arquivos** de código-fonte (em Java ou outros formatos), mas **não pode modificar** arquivos existentes que já foram escritos. [1, 2, 3, 4]

Para que serve?

- **Eliminar código repetitivo (*boilerplate*):** Bibliotecas como o Lombok usam processadores para gerar métodos como `getters` , `setters` , `equals` e `hashCode` "por baixo dos panos" enquanto você compila o projeto. [1]
- **Geração de metadados e classes auxiliares:** Frameworks de persistência como JPA ou QueryDSL usam para gerar metaclasses e arquivos de suporte baseados nas suas entidades. [1]
- **Validação em tempo de compilação:** Permitir que erros ou avisos sejam emitidos antes mesmo de rodar o programa, caso alguma regra de anotação seja violada.

Diferença para anotações em tempo de execução (*Runtime*)

- **Tempo de Compilação (Annotation Processor):** Roda no `javac` . Gera arquivos ou valida o código antes de virar bytecode (.class). É muito usado pelo Lombok ou em geradores de código. [1, 2]
- **Tempo de Execução (*Runtime* / Reflection):** A anotação fica gravada no arquivo `.class` e a JVM (ou frameworks como Spring) lê essas informações enquanto o aplicativo está rodando, usando *Reflection*. [1, 2, 3]

exemplo prático de como criar um Annotation Processor:

Para criar um Annotation Processor prático, vamos construir um gerador de código simples: uma anotação chamada `@BuilderMe` que, ao ser colocada em uma classe, gera automaticamente uma classe Builder para ela durante a compilação.

Para funcionar corretamente no ecossistema Java, a estrutura ideal divide o código em dois submódulos (ou projetos separados):

1. **`processor`**: Contém a anotação e a lógica do processador.
2. **`app`**: O projeto principal que utiliza a anotação para gerar o código.

Passo 1: Criar a Anotação e o Processador (Módulo `processor`)

1. A Anotação (**`BuilderMe.java`**)

Ela deve usar `@Retention(RetentionPolicy.SOURCE)` porque só precisamos dela no momento da compilação.

java

```java
package com.exemplo.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.TYPE) // Alvo: Classes
@Retention(RetentionPolicy.SOURCE) // Descartada após a compilação
public @interface BuilderMe {
}
```

Use o código com cuidado.

2. O Processador (**`BuilderProcessor.java`**)

Esta classe estende `AbstractProcessor` e dita as regras de geração do arquivo de texto contendo o novo código Java.

java

```java
package com.exemplo.processor;

import com.exemplo.annotation.BuilderMe;
import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.Element;
import javax.lang.model.element.TypeElement;
import javax.lang.model.element.VariableElement;
import javax.lang.model.util.ElementFilter;
import javax.tools.JavaFileObject;
import java.io.PrintWriter;
import java.io.Writer;
```

```
import java.util.List;
import java.util.Set;

@SupportedAnnotationTypes("com.exemplo.annotation.BuilderMe")
@SupportedSourceVersion(SourceVersion.RELEASE_17) // Ajuste para a versão do
```

Use o código com cuidado.

3. Registrar o Processador no Compilador

O Java só ativa o processador se ele estiver formalmente registrado.

Dentro do diretório de recursos do módulo `processor`, crie a seguinte estrutura de arquivos: `src/main/resources/META-INF/services/javax.annotation.processing.Processor`

Abra esse arquivo de texto puro e adicione o **nome completo** da classe do processador:

text

```
com.exemplo.processor.BuilderProcessor
```

Use o código com cuidado.

Passo 2: Usar a Anotação no Projeto Principal (Módulo `app`)

Configure as dependências para que o compilador do aplicativo conheça o módulo do processador.

No **`build.gradle`** do seu aplicativo:

groovy

```
dependencies {
    // Importa a anotação para o código principal
    implementation project(':processor')

    // Vincula o processador ao ciclo de compilação do javac
    annotationProcessor project(':processor')
}
```

Use o código com cuidado.

Testando a aplicação (**`Pessoa.java`**)

Basta colocar a anotação na classe. **Atenção:** Como o nosso gerador escreve direto no atributo `this.object.nome`, deixe os atributos com visibilidade de pacote (sem modificador) ou crie setters públicos para evitar erros de compilação.

java

```
package com.exemplo.app;
```

```java
import com.exemplo.annotation.BuilderMe;

@BuilderMe
public class Pessoa {
    String nome;
    int idade;
}
```

Use o código com cuidado.

Executando (`Main.java`)

Assim que você mandar compilar ou rodar o projeto (`gradle build` ou o botão Run da IDE), o compilador gerará a classe `PessoaBuilder` em segundo plano. Você já pode usá-la imediatamente:

java

```java
package com.exemplo.app;

public class Main {
    public static void main(String[] args) {
        // PessoaBuilder foi gerada dinamicamente pelo compilador!
        Pessoa pessoa = new PessoaBuilder()
                            .nome("Rodrigo")
                            .idade(28)
                            .build();

        System.out.println("Criado com sucesso: " + pessoa.nome + ", " +
pessoa.idade + " anos.");
    }
}
```

Use o código com cuidado.

Se inspecionar a pasta de saídas do build do seu projeto (geralmente `build/generated/sources/annotationProcessor`), você verá o arquivo físico `PessoaBuilder.java` criado inteiramente via automação.

Para que um processador de anotações customizado (Annotation Processor) seja reconhecido pelo compilador Java, você deve criar um arquivo de configuração de serviço do SPI (Service Provider Interface).

O nome do arquivo deve ser exatamente **`javax.annotation.processing.Processor`** e ele deve ser colocado dentro do diretório **`META-INF/services/`** dos recursos do seu projeto.

📁 Estrutura do Diretório

Em um projeto padrão (como Maven ou Gradle), a estrutura de pastas deve ficar assim:

text

```
seu-projeto/
└── src/
    └── main/
        └── resources/
            └── META-INF/
                └── services/
                    └── javax.annotation.processing.Processor  <-- O arquivo vai aqui
```

Use o código com cuidado.

📝 Conteúdo do Arquivo

Dentro desse arquivo, você deve colocar apenas o **nome totalmente qualificado (Full Qualified Name)** da classe que implementa o seu processador de anotações (incluindo o pacote).

Por exemplo, se a sua classe se chama `MeuProcessor` e está no pacote `com.exemplo.compiler`, o conteúdo do arquivo será:

text

```
com.exemplo.compiler.MeuProcessor
```

Use o código com cuidado.

Se você tiver mais de um processador no mesmo jar, basta listar um por linha.

💡 Alternativa moderna (Recomendada)

Para evitar ter que criar essa estrutura de pastas e arquivos manualmente, você pode utilizar a biblioteca `AutoService` do Google. Basta adicionar a dependência ao seu projeto e anotar a sua classe:

java

```
@AutoService(Processor.class)
public class MeuProcessor extends AbstractProcessor {
    // Seu código aqui
}
```

Use o código com cuidado.

Isso gerará o arquivo `META-INF/services/javax.annotation.processing.Processor` automaticamente durante a compilação.